EXPERIMENT 10

Applying Performance Optimization Techniques to a Zoho Creator Application

Aim

Reduce unnecessary record retrieval and repeated work, review subform design and explain appropriate indexing in Creator.

Record status

Optimization procedure and local algorithm illustration completed. Live before/after timings and backend index changes are pending.

Concept

Fetch only the records required by a workflow. Prefer aggregate operations when a total is needed. Use bounded batches for larger jobs. Creator backend indexing requires platform support assistance; a unique-field setting should not be reported as a manually created SQL index.

Procedure

1. Identify a slow report or function and record the same operation on the same fixture before changing it. Record dataset size and filtering.

2. Replace fetch-all-and-filter loops with criteria applied in the fetch. Use ID or other suitable selective criteria when available.

3. Review repeated lookups inside loops. Reuse fetched values where correct and avoid API calls for operations supported directly in Deluge.

4. Keep subforms focused on related child rows. Move long independent histories to a child form with a lookup and filter its report.

5. Use a bounded batch for reconciliation. Inspect candidate fields with platform support if indexing is needed. Measure the revised operation with the same fixture and compare correctness as well as timing.

Application design and observations

Technique	Application example
Selective fetch	Only Under Investigation claims eligible for reconciliation
Aggregation	Calculate a report total without a Python-style record loop
Subform review	Keep a small child list; separate long lifecycle history
Batch processing	Existing reconciliation design uses blocks of 100
Indexing	Support-assisted backend work; not performed in this lab

Deluge example

```
// Deluge criteria example; verify names in the application.
claims = Claim_Request_Form[
    Status == "Under Investigation"
];
info claims.count();
```

Local Python illustration: 20,000 fictional records, 600 ID lookups per run, 5 runs. Median scan time: 110.681 ms. Median dictionary lookup: 0.169 ms. Both methods returned identical amounts. Index construction is excluded. These figures illustrate algorithm choice only and are not Creator measurements.

Validation and discussion

Check	Result or required validation
Local lookup equality	Passed across the generated 600-key fixture
Creator result equality	Compare old/new function outputs on the same fixture
Creator timing	Pending; no platform speed-up claimed
Backend index	Not changed; obtain support confirmation if used
Subform redesign	Proposed; schema migration not performed

Result

The optimization design and local lookup comparison are complete. Existing focused batch tests from Experiment 6 support the use of bounded processing, but no new Creator performance benchmark, subform migration or backend index creation was performed.

Viva focus

Explain selective criteria, aggregate queries, subform growth and the difference between local timings and platform evidence.

Reading: Platform performance best practices (official Zoho documentation, reviewed 19 October 2026).

Reading: Fetch collection of records (official Zoho documentation, reviewed 19 October 2026).